RouteWise - Smart Travel Route Optimizer

Project Report

Submitted To

Ms. Kawaljit Kaur

Submitted by

Name – Jagpreet Singh & Chandan Kartik

UID – 26MCI10066 & 26MCI10069

Section - 26MAM-2A

in partial fulfillment for the award of the degree of

Master of Computer Application

Chandigarh University

DECELARATION

I hereby declare that the project titled **"Smart Travel Route Optimizer"** submitted in partial fulfillment for the award of the degree of Master of Computer Applications (AIML) is a record of original work carried out by me under the guidance of my project supervisor.

I further declare that this project has not been submitted previously, in whole or in part, for the award of any degree or diploma in any institution or university. The information presented in this project report is true and correct to the best of my knowledge.

Submitted by :
Jagpreet Singh & Chandan Kartik
Course : Master of Computer Applications - AIML
(MCA) Chandigarh University

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide for their continuous support, valuable guidance, and encouragement throughout the development of this project titled **"Smart Travel Route Optimizer."** Their insights and suggestions greatly helped me in completing this work successfully.

I am also thankful to the faculty members of the Department of Computer Applications for providing the necessary knowledge, resources, and motivation required for this project.

I would like to extend my heartfelt thanks to my institution for offering the facilities and environment needed to carry out this work.

Finally, I am grateful to my family and friends for their constant support, encouragement, and understanding during the entire project duration.

Jagpreet Singh & Chandan Kartik
MCA (AIML)
Chandigarh University

ABSTRACT

The **Smart Travel Route Optimizer** is a system designed to provide efficient and optimized travel routes between different locations. In modern transportation systems, selecting the best route based on factors such as distance, cost, and time is a complex task. This project addresses the problem by applying advanced algorithmic techniques from graph theory and dynamic programming.

The system represents cities and travel paths as a weighted graph, where nodes denote locations and edges represent routes with associated costs, distances, or travel times. To determine the shortest path, **Dijkstra's Algorithm** is used, while **Kruskal's Algorithm** helps in identifying a minimum cost network. Additionally, **Dynamic Programming** techniques are applied for optimizing travel expenses, and sorting algorithms such as Quick Sort and Merge Sort are used to organize routes based on different criteria.

The application allows users to input source and destination locations and provides multiple route options along with comparisons of cost, time, and distance. It also includes graphical analysis to evaluate algorithm performance by plotting time complexity against input size.

This project demonstrates the practical implementation of fundamental algorithms in solving real-world problems and provides a scalable and efficient solution for intelligent travel planning.

TABLE OF CONTENTS

S. No.	Title	Page No.
1	Introduction	6
2	System Requirements	6
3	System Design	7
4	Methodology	8
5	Implementation	10
6	Output	18
7	Advantages & Limitations	20
8	Conclusion	21
9	References	21

INTRODUCTION

In the modern digital era, travel has become an essential part of everyday life, whether for business, education, or tourism. With the increasing number of transportation options and complex route networks, selecting the most efficient path between two locations has become a challenging task. Travelers often face difficulties in choosing routes that minimize travel time, cost, and distance simultaneously.

The **Smart Travel Route Optimizer Using Graph Algorithms** is designed to address these challenges by providing an intelligent system that computes the most optimal travel routes. The project utilizes concepts from graph theory, algorithm design, and dynamic programming to analyze multiple routes and suggest the best possible option based on user requirements.

In this system, cities are represented as nodes and the routes connecting them are represented as edges with associated weights such as distance, cost, or travel time. By applying efficient algorithms like Dijkstra's Algorithm for shortest path calculation and Kruskal's Algorithm for minimum spanning tree construction, the system ensures accurate and optimized route selection. Additionally, dynamic programming techniques help in improving computational efficiency and optimizing travel costs.

The project also focuses on performance analysis by comparing execution times for different input sizes and visualizing the results using graphs. This helps in understanding the efficiency of various algorithms used in the system.

SYSTEM REQUIRMENTS

1. Hardware Requirements

Component	Specification
Processor	Intel Core i3 or higher
RAM	4 GB or more
Storage	500 MB free space
Input Devices	Keyboard

2. Software Requirements

Component	Specification
Operating System	Windows/Linux/macOS
Programming Language	Python
IDE	VS Code / PyCharm
Python Version	Python 3.x

SYSTEM DESIGN

The system follows a simple layered architecture:

- **Input Layer:**
  Accepts user input such as source city, destination city, and route preferences (cost/time/distance).

- **Processing Layer:**
  Core logic where algorithms are applied:
  - Dijkstra's Algorithm for shortest path
  - Kruskal's Algorithm for minimum spanning tree
  - Dynamic Programming for cost optimization
  - Sorting algorithms for ranking routes

- **Output Layer:**
  Displays the optimized route, cost, time, and comparison of multiple routes. Graphs are also generated for performance analysis.

Modules of the System

a. Graph Creation Module
- Represents cities as nodes and routes as edges
- Stores weights such as distance, cost, and time
- Implemented using adjacency list or dictionary in Python

b. Route Optimization Module
- Applies Dijkstra's Algorithm to find shortest path
- Uses Dynamic Programming for optimizing travel cost

c. Network Optimization Module
- Implements Kruskal's Algorithm to generate minimum cost spanning tree

d. Sorting Module
- Uses Quick Sort / Merge Sort to sort routes based on:
  - Cost
  - Distance
  - Time

e. Visualization Module
- Uses matplotlib to plot:
  - Time vs number of nodes
  - Algorithm performance graphs

METHODOLOGY

1. Problem Identification

The first step involves identifying the problem of finding the most efficient travel route between two locations based on multiple parameters such as distance, cost, and time. Traditional methods are inefficient when dealing with large datasets and complex networks.

2. Data Representation
- Cities are represented as **nodes (vertices)**
- Routes between cities are represented as **edges**
- Each edge contains weights such as:
  - Distance
  - Cost
  - Time

The graph is implemented using Python data structures such as dictionaries and lists.

3. Algorithm Selection and Implementation

Different algorithms are used to solve specific parts of the problem:

a. Dijkstra's Algorithm

- Used to find the shortest path between source and destination
- Utilizes a priority queue for efficiency
- Ensures optimal solution for weighted graphs

b. Kruskal's Algorithm

- Used to construct the minimum spanning tree
- Helps in minimizing the total network cost
- Uses disjoint set (union-find) data structure

c. Dynamic Programming

- Applied for cost optimization problems
- Stores intermediate results to avoid recomputation
- Improves performance for repeated calculations

d. Sorting Algorithms (Quick Sort / Merge Sort)

- Used to sort available routes based on:
    - Minimum cost
    - Shortest time
    - Least distance

4. Input Processing

- User provides source and destination cities
- System validates input
- Graph data is loaded or generated dynamically

5. Route Computation

- Dijkstra's Algorithm computes the shortest path
- Multiple routes are analyzed and compared
- Sorting algorithms rank the routes
- Dynamic programming refines cost efficiency

6. Output Generation

- Displays:
    - Best route
    - Total cost
    - Total distance
    - Estimated time
- Provides comparison between alternative routes

7. Performance Analysis

- The system is tested with different input sizes (n)
- Execution time is recorded

- Graphs are plotted (time vs number of nodes) using matplotlib

8. Testing and Validation

- Test cases are created for different scenarios
- Results are verified for correctness
- Edge cases such as no route available are handled

9. Implementation Tools

- Python programming language
- VS Code as development environment
- Libraries like heapq, matplotlib, and random

IMPLEMETATION

main.py

```
from __future__ import annotations
from flask import Flask, render_template, request

from travel_optimizer.algorithms import TravelGraph, quick_sort_routes
from travel_optimizer.data import SAMPLE_EDGES
from travel_optimizer.models import Edge, RouteResult

app = Flask(__name__)
graph = TravelGraph(SAMPLE_EDGES)

def serialize_route(route: RouteResult) -> dict[str, object]:
    return {
        "path": route.path,
        "path_text": " -> ".join(route.path),
        "distance": round(route.total_distance, 1),
        "time": round(route.total_time, 1),
        "cost": round(route.total_cost, 1),
        "stops": route.hops,
        "score_key": route.score_key,
    }

def serialize_edges(edges: list[Edge]) -> list[dict[str, object]]:
    return [
        {
            "source": edge.source,
            "destination": edge.destination,
            "distance": edge.distance,
            "time": edge.time,
```

```
            "cost": edge.cost,
        }
        for edge in edges
    ]

def build_chart_points(routes: list[RouteResult]) -> list[dict[str, object]]:
    return [
        {
            "label": " -> ".join(route.path),
            "nodes": len(route.path),
            "time": round(route.total_time, 1),
            "cost": round(route.total_cost, 1),
        }
        for route in routes
    ]

@app.route("/", methods=["GET", "POST"]) def
index():
   context: dict[str, object] = {
"cities": graph.nodes,
        "selected_source": "",
        "selected_destination": "",
        "budget": "",
        "results": None,
         "error": None,
   }

   if request.method == "POST":
        source = request.form.get("source", "").strip().title()
destination = request.form.get("destination", "").strip().title()
budget_text = request.form.get("budget", "").strip()
max_stops_text = request.form.get("max_stops", "5").strip()

        context["selected_source"]       =       source
context["selected_destination"]    =    destination
context["budget"] = budget_text
        context["max_stops"] = max_stops_text

            mst_edges, total_network_cost = graph.kruskal_minimum_network(weight="cost")

            context["results"] = {
                "shortest_route": serialize_route(shortest_route),
                "cheapest_route": serialize_route(cheapest_route),
                "routes_by_cost": [serialize_route(route) for route in sorted_by_cost[:5]],
                "routes_by_time": [serialize_route(route) for route in sorted_by_time[:5]],
                "budget_route": serialize_route(budget_route) if budget_route else None,
                "budget_checked": bool(budget_text),
                "network_edges": serialize_edges(mst_edges),
                "network_total_cost": round(total_network_cost, 1),
                "chart_points": build_chart_points(sorted_by_time[:5]),
                "route_count": len(all_routes),
            }
```

```
        except ValueError as error:
context["error"] = str(error)

    return render_template("index.html", **context)

if __name__ == "__main__":
    app.run(debug=True)
```

Algorithms.py

```
from __future__ import annotations

import heapq

from dataclasses import dataclass

from .models import Edge, RouteLeg, RouteResult

@dataclass class
DisjointSet:
    parent: dict[str, str]
    rank: dict[str, int]

    @classmethod    def create(cls, items: list[str]) -> "DisjointSet":        return
 cls(parent={item: item for item in items}, rank={item: 0 for item in items})

   def find(self, item: str) -> str:        if self.parent[item]
!=    item:                                  self.parent[item]    =
self.find(self.parent[item])     return self.parent[item]

   def union(self, left: str, right: str) -> bool:
left_root = self.find(left)
        right_root = self.find(right)
if left_root == right_root:
            return False

    def dijkstra(self, source: str, destination: str, weight: str) -> RouteResult:         if
source not in self.adjacency or destination not in self.adjacency:             raise
ValueError("Source or destination city is not available in the network.")         if
source == destination:             return RouteResult(                 path=[source],
legs=[],                total_distance=0.0,                total_time=0.0,
total_cost=0.0,
                score_key=weight,
            )

        distances = {node: float("inf") for node in self.nodes}
        previous: dict[str, Edge | None] = {node: None for node in self.nodes}
distances[source] = 0.0
```

```
        queue: list[tuple[float, str]] = [(0.0, source)]
while queue:
            current_distance, current_node = heapq.heappop(queue)
if current_distance > distances[current_node]:
                continue            if
current_node == destination:
break


            for    edge    in    self.adjacency[current_node]:
edge_weight = getattr(edge, weight)              candidate
= current_distance + edge_weight              if candidate
<                                  distances[edge.destination]:
distances[edge.destination]            =            candidate
previous[edge.destination] = edge
                    heapq.heappush(queue, (candidate, edge.destination))


        if distances[destination] == float("inf"):
            raise ValueError(f"No route found from {source} to {destination}.")


        return self._build_route(previous, source, destination, score_key=weight)

    def kruskal_minimum_network(self, weight: str = "cost") -> tuple[list[Edge], float]:
        sorted_edges = merge_sort_edges(self.edges, key=weight)
        disjoint_set = DisjointSet.create(self.nodes)
        mst: list[Edge] = []
        total_weight = 0.0


        for edge in sorted_edges:              if
disjoint_set.union(edge.source, edge.destination):
                mst.append(edge)
 total_weight += getattr(edge, weight)
 if len(mst) == len(self.nodes) - 1:
 break        return mst, total_weight
    def enumerate_routes(
        self,        source:
str,         destination:
str,
        max_stops: int = 5,     ) -> list[RouteResult]:         if source not in self.adjacency
or destination not in self.adjacency:              raise ValueError("Source or destination
city is not available in the network.")




    while left_index < len(left) and right_index < len(right):         if
getattr(left[left_index], key) <= getattr(right[right_index], key):
merged.append(left[left_index])
            left_index += 1
else:
            merged.append(right[right_index])
            right_index += 1
```

```
    merged.extend(left[left_index:])
merged.extend(right[right_index:])     return
merged
```

test_travel_optimizer.py

```
from travel_optimizer.algorithms import TravelGraph, quick_sort_routes from
travel_optimizer.data import SAMPLE_EDGES

def build_graph() -> TravelGraph:
    return TravelGraph(SAMPLE_EDGES)

def test_shortest_and_cheapest_routes_are_generated() -> None:
graph = build_graph()



    shortest = graph.dijkstra("Delhi", "Mumbai", weight="distance")
cheapest = graph.dijkstra("Delhi", "Mumbai", weight="cost")

    assert shortest.path[0] == "Delhi"
assert shortest.path[-1] == "Mumbai"
assert cheapest.path[0] == "Delhi"
    assert cheapest.path[-1] == "Mumbai"

def test_budget_route_respects_limit() -> None:
graph = build_graph()
    route = graph.best_route_within_budget("Delhi", "Mumbai", budget=7000, optimize="time")

    assert route is not None
    assert route.total_cost <= 7000

def test_kruskal_returns_network() -> None:
    graph = build_graph()
    mst_edges, total_cost = graph.kruskal_minimum_network(weight="cost")



def test_quick_sort_orders_routes_by_cost() -> None:
   graph = build_graph()
   routes = graph.enumerate_routes("Delhi", "Mumbai", max_stops=5)
   sorted_routes = quick_sort_routes(routes, key="cost")
    costs = [route.total_cost for route in sorted_routes]
   assert costs == sorted(costs)
```

Index.html:

```
 <!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>RouteWise - Smart Travel Route Optimizer</title>
   <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<header class="topbar">
   <div class="brand">RouteWise</div>
   <span>Graph-based travel intelligence</span>
</header>

<main class="container">
   <section class="hero">
      <div>
         <p class="eyebrow">SMART TRAVEL PLANNING</p>
         <h1>Smart Travel<br>Route Optimizer</h1>
         <p class="subtitle">Compare routes by distance, travel time and estimated cost.</p>
         <div class="stats">
            <div><strong>{{ cities|length }}</strong><span>Connected Cities</span></div>
            <div><strong>4</strong><span>Core Algorithms</span></div>
         </div>
      </div>

      <form class="planner" method="POST">
         <h2>Plan a Route</h2>
         <label for="source">Source City</label>
         <select id="source" name="source" required>
            <option value="">Select source</option>
            {% for city in cities %}
            <option value="{{ city }}" {% if selected_source == city %}selected{% endif %}>{{ city }}</option>
            {% endfor %}
         </select>

         <label for="destination">Destination City</label>
         <select id="destination" name="destination" required>
            <option value="">Select destination</option>
            {% for city in cities %}
            <option value="{{ city }}" {% if selected_destination == city %}selected{% endif %}>{{ city }}</option>
            {% endfor %}
         </select>

         <label for="budget">Budget (optional, ₹)</label>
         <input id="budget" name="budget" type="number" min="0" step="1"
               placeholder="e.g. 5000" value="{{ budget }}">
```

```
      <label for="max_stops">Maximum Stops</label>
      <input id="max_stops" name="max_stops" type="number" min="1" max="20"
          value="{{ max_stops }}">

      <button type="submit">Optimize Route</button>
    </form>
</section>

{% if error %}
<div class="alert">{{ error }}</div>
{% endif %}

{% if results %}
<section class="section">
    <div class="section-heading">
      <div><p class="eyebrow">YOUR RESULTS</p><h2>Results Dashboard</h2></div>
      <span class="pill">{{ results.route_count }} routes explored</span>
    </div>

    <div class="summary-grid">
      <article class="result-card blue">
        <span>Shortest Route</span>
        <h3>{{ results.shortest_route.path_text }}</h3>
        <p>{{ results.shortest_route.distance }} km · {{ results.shortest_route.time }} hrs · ₹{{ results.shortest_route.cost }}</p>
      </article>
      <article class="result-card orange">
        <span>Cheapest Route</span>
        <h3>{{ results.cheapest_route.path_text }}</h3>
        <p>{{ results.cheapest_route.distance }} km · {{ results.cheapest_route.time }} hrs · ₹{{ results.cheapest_route.cost }}</p>
      </article>
      <article class="result-card green">
        <span>Budget-Optimized Route</span>
        {% if results.budget_route %}
        <h3>{{ results.budget_route.path_text }}</h3>
        <p>{{ results.budget_route.distance }} km · {{ results.budget_route.time }} hrs · ₹{{ results.budget_route.cost }}</p>
        {% elif results.budget_checked %}
        <h3>No route within budget</h3><p>Try increasing your budget.</p>
        {% else %}
        <h3>Budget not provided</h3><p>Enter a budget to see this result.</p>
        {% endif %}
      </article>
    </div>
</section>

<section class="two-columns section">
    <div class="panel">
      <h2>Top Routes by Cost</h2>
      {% for route in results.routes_by_cost %}
      <div class="route-row">
        <strong>{{ route.path_text }}</strong>
        <span>₹{{ route.cost }} · {{ route.distance }} km</span>
```

```
      </div>
      {% else %}<p>No alternative routes found.</p>{% endfor %}
    </div>
    <div class="panel">
      <h2>Top Routes by Time</h2>
      {% for route in results.routes_by_time %}
      <div class="route-row">
        <strong>{{ route.path_text }}</strong>
        <span>{{ route.time }} hrs · ₹{{ route.cost }}</span>
      </div>
      {% else %}<p>No alternative routes found.</p>{% endfor %}
    </div>
</section>

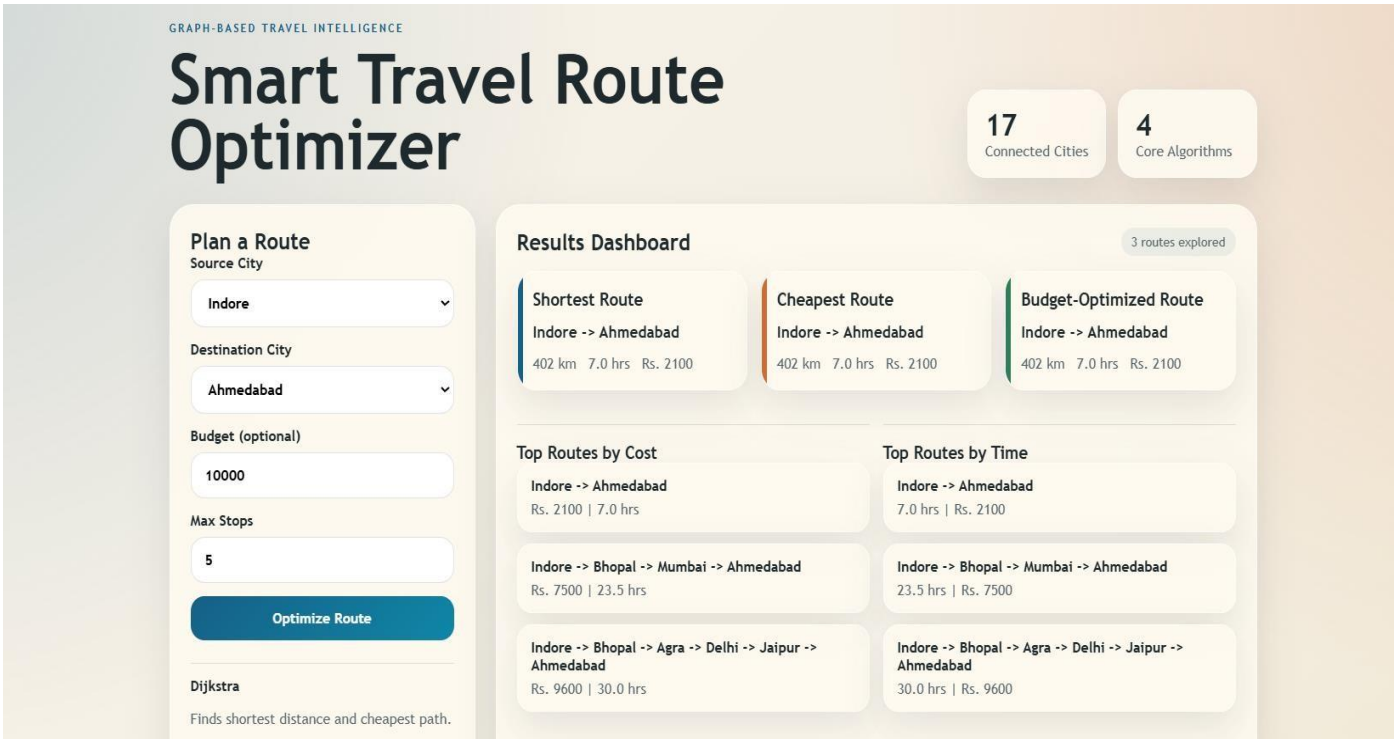
<section class="section panel">
    <h2>Algorithm Overview</h2>
    <div class="algorithm-grid">
      <div><h3>Dijkstra</h3><p>Finds the shortest or lowest-weight route.</p></div>
      <div><h3>Kruskal</h3><p>Builds a minimum-cost network across cities.</p></div>
      <div><h3>Dynamic Programming</h3><p>Budget route selection reuses computed route candidates.</p></div>
      <div><h3>Quick Sort / Merge Sort</h3><p>Ranks alternative routes and network edges.</p></div>
    </div>
</section>

<section class="section panel">
    <h2>Travel Graph: Route Time vs Number of Stops</h2>
    <div class="chart">
      <div class="chart-axis"><span>Travel time (hours)</span></div>
      <div class="bars">
        {% for point in results.chart_points %}
        <div class="bar-item">
          <div class="bar" style="height: {{ [point.time * 5, 12]|max }}px"></div>
          <small>{{ point.nodes }} cities</small>
          <small>{{ point.time }} hrs</small>
        </div>
        {% endfor %}
      </div>
    </div>
    <p class="muted">Illustrative comparison based on the sample route data.</p>
</section>

<section class="section panel">
    <h2>Minimum Cost Network (Kruskal Output)</h2>
    <div class="network-grid">
      {% for edge in results.network_edges %}
      <div class="network-card">
        <strong>{{ edge.source }} to {{ edge.destination }}</strong>
        <span>Cost: ₹{{ edge.cost }}</span>
      </div>
      {% endfor %}
    </div>
```

```
 <p class="total">Total network cost: ₹{{ results.network_total_cost }}</p>
   </section>
   {% endif %}
   <footer>RouteWise · Smart Travel Route Optimizer · Sample data is simulated</footer>
</main>
</body>
</html>
```

OUTPUT:



Kruskal

Builds a minimum-cost network across all cities.

Dynamic Programming

Finds the best route under a budget cap.

Quick Sort

Ranks alternate routes by cost and time.

Travel Graph: Time vs Nodes

Browser-rendered chart

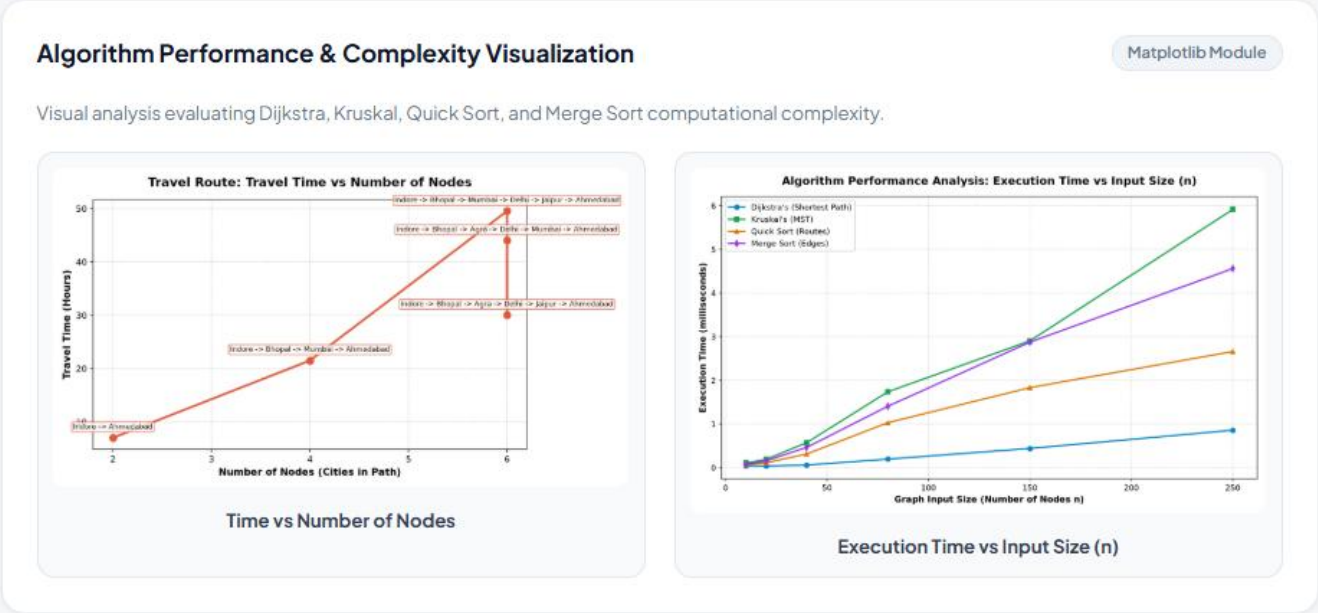


Minimum Cost Network

Kruskal output

Chandigarh to Shimla Cost: Rs. 900	**Mumbai to Pune** Cost: Rs. 950	**Bhopal to Indore** Cost: Rs. 1200	**Chandigarh to Amritsar** Cost: Rs. 1300
Delhi to Agra Cost: Rs. 1400	**Varanasi to Patna** Cost: Rs. 1400	**Delhi to Chandigarh** Cost: Rs. 1500	**Udaipur to Ahmedabad** Cost: Rs. 1500
Delhi to Jaipur Cost: Rs. 1600	**Lucknow to Varanasi** Cost: Rs. 1700	**Agra to Lucknow** Cost: Rs. 1800	**Indore to Ahmedabad** Cost: Rs. 2100
Jaipur to Udaipur Cost: Rs. 2200	**Pune to Goa** Cost: Rs. 2400	**Ahmedabad to Mumbai** Cost: Rs. 2700	**Patna to Kolkata** Cost: Rs. 3000

Total network cost: Rs. 27650.0



ADVANTAGES AND LIMITATIONS

ADVANTAGES

- Provides efficient route optimization using algorithms like Dijkstra's and Kruskal's
- Considers multiple factors such as cost, time, and distance
- Faster computation using dynamic programming
- Sorts routes for better comparison
- Useful for real-world travel planning

LIMITATIONS

- Uses static or simulated data
- Does not include real-time factors like traffic or weather
- Limited user interface in basic implementation
- Performance may reduce for very large datasets

Conclusion

The **Smart Travel Route Optimizer** successfully demonstrates the practical application of important algorithmic techniques in solving real-world problems. By using algorithms such as Dijkstra's for shortest path, Kruskal's for minimum cost network, dynamic programming for optimization, and sorting techniques for comparison, the system efficiently determines the best travel routes.

The project provides an effective solution for selecting routes based on cost, time, and distance, while also helping in understanding the performance of different algorithms. It enhances both theoretical knowledge and practical skills in algorithm design and implementation.

Overall, the system is efficient, scalable, and can be further improved by integrating real-time data and advanced features in the future.

REFERENCES

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
- Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, *Data Structures and Algorithms*, Pearson.
- GeeksforGeeks – Algorithms and Data Structures Tutorials.
- W3Schools – Web Development and Programming Tutorials.
- Python Official Documentation – https://docs.python.org
- Matplotlib Documentation – https://matplotlib.org
- NetworkX Documentation – https://networkx.org